

SECTION 1 — SYSTEM OVERVIEW

SAMS is a web-based application for Bhatia General Contracting Co. (L.L.C.) — BGCC, Dubai UAE — that manages the full lifecycle of subcontract agreements (SCA). It controls creation from master templates, multi-level approval, PDF generation, subcontractor communication, and permanent archival.

This document is the single source of truth for building the system. Give it to any AI coding assistant (Cursor, GitHub Copilot, Claude) at the start of each task.

1.1 The Three Core Documents

Every SCA is a package of three master documents combined into one PDF:

File	Document	Purpose
FILE 01	Form of Subcontract Agreement	Short ~4 pages. Party details, contract price, signature block. 8 [Insert] fields.
FILE 02	Conditions of Subcontract Agreement	15 sections, 40+ clauses. Full legal terms. 13 [Insert] fields spread across clauses.
FILE 03	Appendix to the Subcontract Agreement	3-column summary table. Maps clause number → entered value. 23 rows, all variable.

Admin fills [Insert] placeholders only. Legal boilerplate is never retyped. The Appendix auto-populates from the values Admin enters in the Form and Conditions.

1.2 Users and Roles (15 Users Total)

Role	Count	Permissions
Admin	~2	Full system access: create/edit agreements, manage masters, generate PDFs, manage users and roles
Project Director	~1	Review + approve/return agreements, view/edit all comments
Accounts Department	~2	Review financial clauses, approve/return
Operation Manager	~2	Review operational clauses, approve/return, manage subcontractor comment responses
General Manager	~1	Final approval, view AI executive summary

NOTE: BGCC manages users themselves — Admin can add/edit/deactivate any user and change their role at any time through the system UI. No developer involvement needed.

1.3 Confirmed Business Rules (All 7 Client Confirmations)

#	Rule	Detail + Status
1	Sections + Clauses	Admin CAN add/delete sections and clauses from master templates at any time. Fully flexible. ✓ IN PLAN
2	[Insert] Placeholders	Admin CAN change, add, or remove any [Insert] placeholder field in the future. Field catalog is fully editable. ✓ IN PLAN
3	Legal Text Editing	Admin role (and optionally a designated 'Legal Editor' role) CAN edit legal content of master templates. Regular users cannot. Free-text editing of legal content IS allowed — for authorized roles only. ✓ UPDATED
4	Predefined Fields Flexibility	The predefined fields catalog is fully editable by Admin: add new fields, remove fields, change input types, reorder. ✓ CONFIRMED IN PLAN
5	Reference Number Control	BGCC Admin controls the reference number format and can set/override the reference number for any agreement. System suggests auto-generated number but Admin can modify before finalizing. ✓ UPDATED
6	User Management by BGCC	Admin can add, edit, deactivate, and change roles for all 15 users directly in the system. No developer needed. ✓ CONFIRMED IN PLAN
7	Archive Status Fields	Two new fields added to archive: Current Status (7 values) + Status Updated On (date of last status change). ✓ NEW — ADDED

SECTION 2 — COMPLETE [INSERT] FIELDS CATALOG

Every [Insert...] placeholder identified from the three actual client documents. These are the **ONLY** fields Admin fills. All other content is master template text.

2.1 Form of Subcontract Agreement — 8 Fields

ID	Clause	Placeholder / Label	Input Type + Notes
F01	Cover	Day of signing (blank line)	Date picker — auto-formats as 'DD Day of Month YYYY'
F02	Between (1st Party)	[Insert Name] — Subcontractor company name	Text — feeds into A01 in Appendix automatically
F03	Between (1st Party)	[Insert PO] — Subcontractor PO Box number	Text — feeds into A06 address fields
F04	Between (1st Party)	[TL Nr.] — Trade Licence Number	Text
F05	Preamble	[Insert Employer Name] — the Employer entity	Text — feeds into A02 in Appendix
F06	Preamble	[Insert Project Name / Details]	Textarea — feeds into A04 in Appendix
F07	Preamble	[Insert Project Location]	Text — feeds into A04 extended
F08	Clause 2 — Price	[Insert Amount] — Subcontract Price in AED	Number (AED format) — feeds into A07 in Appendix

2.2 Conditions of Subcontract Agreement — 13 Fields

ID	Clause	Label	Input Type	Default
C01	2.2	Scope of Works — detailed description	Rich textarea	None — required
C02	3.3	Quantities Type	Dropdown: Lump Sum / Re-measurable / Mixed	None
C03	3.4.1	Advance Payment Amount (AED)	Number (AED)	10% of F08
C04	3.4.1	Advance Payment release condition	Text	None

C05	3.4.6	Interim Payment — days after application	Number (integer)	None — required
C06	3.4.7	1st Half Retention release — days after event	Number (integer)	None
C07	3.4.7	2nd Half Retention release — days after event	Number (integer)	None
C08	4.3	Time for Completion — overall project	Text (e.g. '18 months')	None — required
C09	4.3	Milestone table (activity name + date per row)	Dynamic table — add/remove rows	None
C10	5	Defects Liability Period — months	Number (integer)	12 months
C11	6.2	Rate of Liquidated Damages (AED per calendar day)	Number (AED)	None — required
C12	10.1	Insurance documents submission deadline — days from commencement	Number (integer)	None
C13	13.3	Dispute Resolution jurisdiction	Text / dropdown	None — required

2.3 Appendix — Additional Fields (Not Already Covered Above)

ID	Clause	Row Label	Input Type	Source/Default
A01	1.1	The Subcontractor	Text	Auto from F02
A02	1.1	The Employer	Text	Auto from F05
A03	1.1	Engineer/Consultant/ Subconsultant	Text	Manual entry
A04	1.1	The Project	Textarea	Auto from F06+F07
A05	1.6	Main Contractor communications address	Multi-field: Attention, Position Title, Address, Fax, Email	BGCC standard — pre-filled, editable
A06	1.6	Subcontractor communications address	Multi-field: Attention, Position Title, Address, Fax, Email	Partly from F02/F03
A07	3.1	Subcontract Price (AED)	Number	Auto from F08
A08	3.3	Subcontract Quantities Type	Dropdown	Auto from C02
A09	3.4.1	Advance Payment Amount (AED)	Number	Auto from C03
A10	3.4.2	Performance Security (%)	Number + % — editable	10% default

A11	3.4.1	Advance Payment release condition	Text	Auto from C04
A12	3.4.6	Interim Payment days	Number	Auto from C05
A13	3.4.7	1st Half Retention release	Text	Auto from C06
A14	3.4.7	2nd Half Retention release	Text	Auto from C07
A15	4.1	Commencement Date	Date picker	Manual entry — required
A16	4.3(a)	Time for Completion — Project	Text	Auto from C08
A17	4.3(a)	Time for Completion — Subcontract Works	Text	Manual if different from C08
A18	4.3(b)	Milestones MS1, MS2 (and more)	Dynamic table — auto from C09	Auto from C09
A19	5	Defects Liability Period	Number (months)	Auto from C10
A20	6.2	Rate of Liquidated Damages (AED/day)	Number	Auto from C11
A21	6.2	Maximum Liquidated Damages (%)	Number + %	10% default, editable
A22	10.1	Insurance submission deadline (days)	Number	Auto from C12
A23	13.3	Dispute Resolution Jurisdiction	Text	Auto from C13

SECTION 3 — DATABASE SCHEMA

3.1 Key Tables

users

id UUID PK | name TEXT | email TEXT UNIQUE | password_hash TEXT | role ENUM(admin,project_director,accounts,operation_manager,gm) | is_active BOOL | created_at TIMESTAMP | last_login TIMESTAMP

master_templates

id UUID PK | type ENUM(form,conditions,appendix) | version_number TEXT | version_date DATE | content_html TEXT | is_active BOOL | created_by UUID FK→users | notes TEXT | created_at TIMESTAMP

master_fields

id UUID PK | template_id UUID FK | field_id TEXT (F01,C01,A01...) | clause_number TEXT | field_label TEXT | input_type ENUM(text,number,date,dropdown,textarea,multifield,table) | default_value TEXT | is_required BOOL | auto_source_field_id TEXT | appendix_row_label TEXT | appendix_clause_ref TEXT | show_in_appendix BOOL | sort_order INT

NOTE: master_fields is fully Admin-editable: add new fields, delete fields, change input types, reorder. This covers client confirmations #1, #2, and #4.

projects

id UUID PK | project_name TEXT | project_code TEXT UNIQUE | project_location TEXT | employer_name TEXT | engineer_name TEXT | created_by UUID | created_at TIMESTAMP

subcontractors

id UUID PK | company_name TEXT | po_box TEXT | trade_licence_no TEXT | contact_person TEXT | email TEXT | phone TEXT | address TEXT | created_at TIMESTAMP

agreements

id UUID PK | reference_number TEXT UNIQUE (BGCC-controlled, editable before GM approval) | project_id UUID FK | subcontractor_id UUID FK | form_version_id UUID FK | conditions_version_id UUID FK | appendix_version_id UUID FK | current_status ENUM(under_drafting, under_internal_review, draft_forwarded_to_subcontractor, under_subcontractor_review, under_subcontractor_signature, under_bgcc_revision, under_gm_signature, completed) | status_updated_on TIMESTAMP | gm_approval_date DATE | execution_date DATE | is_executed BOOL | created_by UUID | created_at TIMESTAMP | updated_at TIMESTAMP

NOTE: current_status uses the 7+1 values confirmed by client. status_updated_on auto-updates on every status change. This covers client confirmation #7.

`agreement_field_values`

id UUID PK | agreement_id UUID FK | field_id TEXT | entered_value TEXT |
is_modified_from_default BOOL | modification_reason TEXT | entered_by UUID | entered_at
TIMESTAMP

`appendix_config`

id UUID PK | agreement_id UUID FK | field_id TEXT | show_in_appendix BOOL | admin_extra_note
TEXT | sort_order INT | last_modified_by UUID | last_modified_at TIMESTAMP

`workflow_steps`

id UUID PK | agreement_id UUID FK | step_name TEXT | step_order INT | role_required ENUM |
status ENUM(pending,approved,returned,skipped) | assigned_user_id UUID FK | acted_by UUID |
acted_at TIMESTAMP | created_at TIMESTAMP

`workflow_comments`

id UUID PK | workflow_step_id UUID FK | agreement_id UUID FK | original_author_id UUID FK |
last_edited_by_id UUID FK | comment_text TEXT | clause_reference TEXT | status
ENUM(open,in_discussion,resolved) | created_at TIMESTAMP | last_edited_at TIMESTAMP

`comment_edit_history`

id UUID PK | comment_id UUID FK | edited_by UUID FK | previous_text TEXT | new_text TEXT |
edited_at TIMESTAMP

`comments_resolution_sheets`

id UUID PK | agreement_id UUID FK | subcontractor_comment TEXT | clause_reference TEXT |
ai_suggested_response TEXT | pd_response TEXT | om_response TEXT | final_response TEXT |
last_edited_by UUID | is_resolved BOOL | created_at TIMESTAMP | updated_at TIMESTAMP

`ai_reviews`

id UUID PK | agreement_id UUID FK | workflow_step_id UUID | review_type
ENUM(comparison,risk,summary,response_suggestion,validation) | risk_score INT(1-5) |
findings_json JSONB | generated_at TIMESTAMP | model_used TEXT

deviation_reports

id UUID PK | agreement_id UUID FK | generated_by UUID | generated_at TIMESTAMP |
report_data_json JSONB | pdf_path TEXT

pdf_outputs

id UUID PK | agreement_id UUID FK | pdf_type ENUM(draft,final,executed) | file_path TEXT |
generated_at TIMESTAMP | generated_by UUID | watermark_type TEXT

audit_log

id UUID PK | user_id UUID FK | action TEXT | entity_type TEXT | entity_id UUID | old_value_json
JSONB | new_value_json JSONB | ip_address TEXT | created_at TIMESTAMP

SECTION 4 — COMPLETE WORKFLOW

4.1 Agreement Status Values (8 States)

Status Value	Description
under_drafting	Admin is creating/editing the agreement. Not yet submitted for review.
under_internal_review	In the approval chain (PD → Accounts → OM → GM). Status_updated_on = latest step date.
draft_forwarded_to_subcontractor	Admin sent draft PDF to subcontractor after internal approval.
under_subcontractor_review	Subcontractor is reviewing. Awaiting their response.
under_subcontractor_signature	Final version sent to subcontractor for signature.
under_bgcc_revision	Subcontractor comments received. Admin revising.
under_gm_signature	Revised version approved by OM+GM. Under BGCC execution.
completed	Signed by both parties. Record locked. Execution date recorded.

4.2 Approval Chain (Part A)

Triggered when Admin submits a draft agreement. Creates 4 workflow_steps automatically.

- Step 1 — Project Director: Reviews full agreement + Deviation Report + AI risk flags. Can add/edit comments. Approve → Step 2. Return (mandatory comment) → Admin.
- Step 2 — Accounts Department: Reviews financial clauses. Sees PD comments (can edit). Approve → Step 3. Return → Admin.
- Step 3 — Operation Manager: Reviews operational clauses. Sees all previous comments (can edit any). Approve → Step 4. Return → Admin.
- Step 4 — General Manager: Reviews everything + AI executive summary. Sees all comments (can edit any). Approve → Admin generates PDF. Return → Admin.

NOTE: When any reviewer RETURNS: they must enter a comment (linked to optional clause ref). Admin sees the comment and the specific field. Admin revises → resubmits → SAME step reactivates (not full chain).

NOTE: Collaborative comments: all reviewers see ALL comments from all steps. Any reviewer can edit any previous comment. Every edit stored in comment_edit_history with who/when/what changed.

4.3 Subcontractor Response Cycle (Part B)

- Admin records: Signed → Upload scanned copy → execution_date recorded → is_executed=true → LOCKED → Archive updated → Status: completed
- Admin records: Comments → Create Comments Resolution Sheet (one row per comment + clause ref + AI suggested response)
- Project Director reviews sheet → updates/adds to responses
- Operation Manager reviews sheet → confirms or edits responses (can edit PD's responses)
- Admin incorporates ALL agreed responses → revises agreement fields
- OM approves revised → GM approves revised → Admin sends → Status: under_subcontractor_signature
- Repeat if new comments received

SECTION 5 — AI FEATURES

All AI features use OpenAI GPT-4o API. Results are cached in Redis (avoid re-running). AI is assistive only — no action is automatic. Every AI output requires human confirmation.

Feature	Trigger	Output
---------	---------	--------

Clause Comparison	On agreement submission	Structured diff: every field vs master default. Clause Default Entered Change type.
Risk Detection	After clause comparison	Risk score 1–5 per modified clause + plain-English legal/financial risk explanation.
Role-Specific Summary	When reviewer opens agreement	Financial focus for Accounts. Operational for OM. 5-line executive brief for GM.
Response Generator	When Comments Resolution Sheet created	Suggested contractual response per subcontractor comment. Reviewed by PD and OM.
Revision Validation	After Admin revises following comments	Confirms every comment was addressed before resubmission.

SECTION 6 — TECHNICAL STACK + ALL PACKAGES

6.1 Architecture Decision

15 users. Single Hostinger VPS KVM 4 (4 vCPU, 16GB RAM). NO Docker. Direct deployment with Nginx + Supervisor. Python FastAPI backend. React TypeScript frontend. PostgreSQL + Redis. No paid services except OpenAI API.

6.2 Backend — Python Packages (requirements.txt)

```
fastapi==0.115.0

uvicorn[standard]==0.30.0

sqlalchemy==2.0.35

alembic==1.13.3

psycopg2-binary==2.9.9

python-jose[cryptography]==3.3.0

passlib[bcrypt]==1.7.4
```

redis==5.0.8

pydantic-settings==2.5.2

python-multipart==0.0.12

jinja2==3.1.4

weasyprint==62.3

openai==1.51.0

tiktoken==0.7.0

slowapi==0.1.9

python-dotenv==1.0.1

aiosmtplib==3.0.1

apscheduler==3.10.4

httpx==0.27.2

python-docx==1.1.2

openpyxl==3.1.5

Install command:

```
pip install -r requirements.txt
```

6.3 Frontend – Node Packages (package.json)

react@18.3.1 + react-dom@18.3.1

react-router-dom@6.26.2

axios@1.7.7

zustand@4.5.5

@tanstack/react-query@5.56.2

```
@tiptap/react@2.6.6 + @tiptap/starter-kit@2.6.6 + @tiptap/extension-placeholder  
  
tailwindcss@3.4.13 + @tailwindcss/forms  
  
lucide-react@0.447.0  
  
react-hook-form@7.53.0  
  
zod@3.23.8  
  
@hookform/resolvers@3.9.0
```

Install command:

```
npm create vite@latest frontend -- --template react-ts  
  
cd frontend && npm install react-router-dom axios zustand @tanstack/react-query @tiptap/  
react @tiptap/starter-kit @tiptap/extension-placeholder tailwindcss @tailwindcss/forms  
lucide-react react-hook-form zod @hookform/resolvers
```

6.4 VPS Setup Commands (Run Once on Hostinger VPS)

```
sudo apt update && sudo apt upgrade -y  
  
sudo apt install -y python3.11 python3.11-venv python3-pip postgresql postgresql-contrib  
redis-server nginx supervisor git curl nodejs npm  
  
# WeasyPrint system dependencies (for PDF generation):  
  
sudo apt install -y python3-cffi python3-brotli libpango-1.0-0 libpangoft2-1.0-0 libffi-dev  
libcairo2 libgdk-pixbuf2.0-0 shared-mime-info  
  
# PostgreSQL setup:  
  
sudo -u postgres psql -c "CREATE DATABASE sams_db;"  
  
sudo -u postgres psql -c "CREATE USER sams_user WITH PASSWORD 'STRONG_PASSWORD';"  
  
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE sams_db TO sams_user;"  
  
# SSL certificate:  
  
sudo certbot --nginx -d yourdomain.com
```

```
# Python virtual environment:

python3.11 -m venv /var/www/sams/backend/venv

source /var/www/sams/backend/venv/bin/activate

pip install -r /var/www/sams/backend/requirements.txt
```

SECTION 7 — FOLDER STRUCTURE

sams/

└─ backend/

| └─ main.py # FastAPI app entry point

| └─ config.py # Settings (DB URL, OpenAI key, etc.)

| └─ database.py # SQLAlchemy engine + session

| └─ models/

| | └─ user.py

| | └─ master.py # master_templates + master_fields

| | └─ agreement.py # agreements + field_values + appendix_config

| | └─ workflow.py # steps + comments + edit_history

| | └─ resolution.py # comments_resolution_sheets

| | └─ ai_review.py

```
| | └─ audit.py

| └─ schemas/                # Pydantic request/response models

| └─ routers/

| | └─ auth.py

| | └─ users.py              # BGCC manages own users – full CRUD

| | └─ masters.py            # template versions + field catalog

| | └─ agreements.py          # creation wizard + field values

| | └─ workflow.py            # approval chain + comments

| | └─ resolution.py          # subcontractor comments cycle

| | └─ ai.py                  # all AI endpoints

| | └─ pdf.py                 # PDF generation + deviation report

| | └─ archive.py             # project + subcontractor archive

| └─ services/

| | └─ workflow_engine.py     # state machine

| | └─ ai_service.py          # OpenAI calls + Redis cache

| | └─ pdf_service.py         # WeasyPrint

| | └─ email_service.py       # SMTP via aiosmtplib

| | └─ audit_service.py

| └─ templates/              # Jinja2 HTML for PDF
```

```
| | | └─ cover_page.html

| | | └─ form_of_agreement.html

| | | └─ conditions.html

| | | └─ appendix.html

| | | └─ deviation_report.html

| | └─ base_pdf.css

| └─ migrations/                # Alembic

| └─ requirements.txt

└─ frontend/

    └─ src/

        └─ pages/

            | | └─ Dashboard.tsx

            | | └─ AgreementCreate.tsx # Multi-step wizard

            | | └─ AgreementDetail.tsx

            | | └─ MasterTemplates.tsx # Admin: versions + field catalog

            | | └─ WorkflowReview.tsx  # Reviewers: approve/return/comment

            | | └─ CommentsResolution.tsx

            | | └─ Archive.tsx          # Project + subcontractor views

            | | └─ UserManagement.tsx   # Admin: BGCC manages own users
```

```

|       |   └─ Reports.tsx

|       └─ components/

|       |   └─ FieldInput.tsx      # Dynamic input by field type

|       |   └─ AppendixBuilder.tsx

|       |   └─ AIReviewPanel.tsx

|       |   └─ WorkflowTimeline.tsx

|       |   └─ CommentThread.tsx   # Collaborative comment view

|       |   └─ DeviationReport.tsx

|       └─ api/                    # Axios API clients per domain

└─ uploads/                        # PDF storage on VPS

└─ nginx/sams.conf

└─ scripts/

    └─ setup.sh                    # One-command VPS setup

    └─ backup.sh                   # Daily PostgreSQL backup

```

SECTION 8 — TASK BREAKDOWN + AI PROMPTS

Each task below is self-contained. Give the AI assistant this full document + the specific task prompt. Tasks are ordered by dependency — complete in sequence.

TASK 01	VPS Setup + Project Scaffolding
Phase	Week 1, Day 1
Stack	bash, Python 3.11, FastAPI, PostgreSQL, Redis, Nginx, Supervisor
Description	Configure the Hostinger VPS for production. Create the full project folder structure. Initialize FastAPI app with config, database connection, and health check endpoint. Set up PostgreSQL database with sams_db. Configure Nginx as reverse proxy. Configure Supervisor to manage the FastAPI process.
Files to create	<code>backend/main.py</code> <code>backend/config.py</code> <code>backend/database.py</code> <code>nginx/sams.conf</code> <code>supervisord.conf</code> <code>scripts/setup.sh</code>
AI Prompt	You are building SAMS (Subcontract Agreement Management System) for BGCC Dubai. TASK 01: Set up the production server and project structure. Create a FastAPI app at <code>backend/main.py</code> with: (1) CORS configured for the frontend domain, (2) a / health endpoint returning <code>{status: ok, version: 1.0}</code> , (3) SQLAlchemy async engine connected to PostgreSQL using <code>DATABASE_URL</code> from <code>.env</code> , (4) Alembic configured in <code>alembic.ini</code> . Create <code>backend/config.py</code> using pydantic-settings to load: <code>DATABASE_URL</code> , <code>REDIS_URL</code> , <code>OPENAI_API_KEY</code> , <code>JWT_SECRET_KEY</code> , <code>SMTP_HOST</code> , <code>SMTP_USER</code> , <code>SMTP_PASS</code> , <code>FRONTEND_URL</code> . Create <code>nginx/sams.conf</code> that proxies <code>/api/*</code> to <code>localhost:8000</code> and serves the React build at <code>/</code> . No Docker. Direct deployment only.

TASK 02	Database Models + Migrations
Phase	Week 1, Day 1–2
Stack	SQLAlchemy, Alembic, PostgreSQL
Depends on	TASK 01
Description	Create all SQLAlchemy ORM models matching the schema in Section 3. Run Alembic to generate and apply the initial migration. Seed the <code>master_fields</code> table with all 27 [Insert] fields from Section 2.
Files to create	<code>backend/models/user.py</code> <code>backend/models/master.py</code> <code>backend/models/agreement.py</code> <code>backend/models/workflow.py</code> <code>backend/models/resolution.py</code> <code>backend/models/ai_review.py</code> <code>backend/models/audit.py</code> <code>backend/migrations/versions/001_initial.py</code> <code>backend/scripts/seed_fields.py</code>

AI Prompt

You are building SAMS. TASK 02: Create all SQLAlchemy models. Each model file should import Base from database.py. Create: (1) users table with id(UUID PK), name, email(unique), password_hash, role(enum: admin/project_director/accounts/operation_manager/gm), is_active(bool default true), created_at, last_login. (2) master_templates: id, type(enum: form/conditions/appendix), version_number, version_date, content_html, is_active, created_by(FK users), notes, created_at. (3) master_fields: id, template_id(FK), field_id(text, e.g. F01), clause_number, field_label, input_type(enum: text/number/date/dropdown/textarea/multifield/table), default_value, is_required(bool), auto_source_field_id(text nullable), appendix_row_label, appendix_clause_ref, show_in_appendix(bool default true), sort_order(int). (4) agreements: id, reference_number(text unique), project_id(FK), subcontractor_id(FK), form_version_id(FK), conditions_version_id(FK), current_status(enum: under_drafting/under_internal_review/draft_forwarded_to_subcontractor/under_subcontractor_review/under_subcontractor_signature/under_bgcc_revision/under_gm_signature/completed), status_updated_on(timestamp auto-updates), gm_approval_date, execution_date, is_executed(bool default false), created_by(FK), created_at, updated_at. (5) agreement_field_values: id, agreement_id(FK), field_id(text), entered_value(text), is_modified_from_default(bool), modification_reason(text), entered_by(FK), entered_at. (6) appendix_config: id, agreement_id(FK), field_id(text), show_in_appendix(bool), admin_extra_note(text), sort_order(int), last_modified_by(FK), last_modified_at. (7) workflow_steps: id, agreement_id(FK), step_name, step_order(int), role_required(enum), status(enum: pending/approved/returned/skipped), assigned_user_id(FK), acted_by(FK nullable), acted_at(timestamp nullable). (8) workflow_comments: id, workflow_step_id(FK), agreement_id(FK), original_author_id(FK), last_edited_by_id(FK nullable), comment_text, clause_reference(text nullable), status(enum: open/in_discussion/resolved), created_at, last_edited_at. (9) comment_edit_history: id, comment_id(FK), edited_by(FK), previous_text, new_text, edited_at. (10) comments_resolution_sheets, ai_reviews, deviation_reports, pdf_outputs, audit_log — see Section 3 of the SAMS context document for full field definitions. Generate Alembic migration. Create seed script for 27 master_fields from Section 2 of the context document.

TASK 03	Authentication + User Management
Phase	Week 1, Day 2
Stack	FastAPI, python-jose, passlib, JWT
Depends on	TASK 02
Description	JWT-based authentication with login, token refresh, and logout. Role-based middleware to protect all routes. User CRUD endpoints (Admin only) — BGCC manages their own users. No external auth service needed.
Files to create	backend/routers/auth.py backend/routers/users.py backend/services/auth_service.py backend/middleware/rbac.py

AI Prompt

You are building SAMS. TASK 03: Build authentication and user management. (1) POST /api/auth/login: accepts {email, password}, verifies against users table using passlib bcrypt, returns {access_token, refresh_token, user: {id, name, email, role}}. Access token expires in 8 hours, refresh token in 30 days. Both are JWT signed with JWT_SECRET_KEY. (2) POST /api/auth/refresh: accepts {refresh_token}, returns new access_token. (3) POST /api/auth/logout: invalidates refresh token. (4) Create RBAC middleware: get_current_user() dependency that decodes JWT and returns user. Create require_role(*roles) dependency factory. (5) User management (Admin only — /api/users/): GET /api/users/ (list all), POST /api/users/ (create), PUT /api/users/{id} (edit name/email/role), PATCH /api/users/{id}/deactivate, PATCH /api/users/{id}/activate. Admin can change any user role at any time. (6) All user management actions log to audit_log with action=user_created/user_edited/user_deactivated etc.

TASK 04	Master Template Management
Phase	Week 1, Day 3–4
Stack	FastAPI, SQLAlchemy, TipTap (frontend)
Depends on	TASK 02 → TASK 03
Description	Versioned master template management. Admin can add/delete sections, add/delete clauses, add/delete/modify [Insert] fields. Every save creates a new version. Previous versions preserved.
Files to create	<code>backend/routers/masters.py</code> <code>backend/services/master_service.py</code> <code>frontend/src/pages/MasterTemplates.tsx</code> <code>frontend/src/components/FieldCatalog.tsx</code>
AI Prompt	You are building SAMS. TASK 04: Build master template management. (1) GET /api/masters/ — list all templates grouped by type (form/conditions/appendix) with version history. (2) GET /api/masters/{id} — get full template content + all master_fields for that version. (3) POST /api/masters/ — create new version of a template type. Body: {type, version_number, version_date, content_html, notes}. Auto-sets is_active=false for previous versions of same type. Logs to audit_log. (4) GET /api/masters/fields/{template_id} — get all master_fields for a template, ordered by sort_order. (5) POST /api/masters/fields/ — add new field to a template. Body: {template_id, field_id, clause_number, field_label, input_type, default_value, is_required, auto_source_field_id, appendix_row_label, appendix_clause_ref, show_in_appendix, sort_order}. (6) PUT /api/masters/fields/{field_id} — update field properties. (7) DELETE /api/masters/fields/{field_id} — remove field. (8) PUT /api/masters/fields/reorder — bulk reorder. All field operations are Admin-only and logged. Note: Admin CAN edit the HTML content of master templates (legal text editing is allowed for admin role — client confirmation #3). Build a React page with: version history list, TipTap editor for HTML content, and a field catalog table showing all 27 fields from Section 2 with add/edit/delete buttons.

TASK 05	Agreement Creation Wizard
Phase	Week 1, Day 5–7
Stack	FastAPI, React, React Hook Form, Zod
Depends on	TASK 03 → TASK 04
Description	Multi-step creation wizard. Admin fills ONLY the [Insert] fields (never edits legal text). System auto-populates dependent fields. Reference number suggested by system but Admin can override before GM approval.
Files to create	<code>backend/routers/agreements.py</code> <code>backend/services/agreement_service.py</code> <code>frontend/src/pages/AgreementCreate.tsx</code> <code>frontend/src/components/FieldInput.tsx</code>
AI Prompt	You are building SAMS. TASK 05: Build the agreement creation wizard. The wizard has 5 steps: STEP 1: Select or create Project (name, code, location, employer, engineer) and Subcontractor (name, PO box, trade licence, contact, email). STEP 2: Fill Form fields F01-F08 (see Section 2.1 of context document for full list). Each field rendered based on its input_type from master_fields. Date picker for F01, text for F02-F07, number+AED for F08. STEP 3: Fill Conditions fields C01-C13. Render appropriate input per input_type. C09 is a dynamic table (add/remove milestone rows). C02 is a dropdown (Lump Sum/Re-measurable/Mixed). STEP 4: Appendix Builder. Show all appendix rows (A01-A23). Auto-populated fields show their values (read-only with source indicator). Admin can toggle show/hide per row. Admin can add extra_note per row. Admin can reorder rows. STEP 5: Review all entered values. Show which fields are modified from master default (highlight in amber). Admin can set/override reference number (system suggests SAG-[PROJECT_CODE]-[YEAR]-[SEQ]). Save as draft. API: POST /api/agreements/ (create draft), PUT /api/agreements/{id}/fields (update field values), POST /api/agreements/{id}/submit (submit for review — creates 4 workflow_steps). Auto-population: when F02 changes, update A01. When F05 changes, update A02. When F08 changes, update A07 and recalculate C03 default. All entered values saved to agreement_field_values. Modified-from-default flag auto-set when entered value != master_fields.default_value.

TASK 06	PDF Generation Engine
Phase	Week 2, Day 8–9
Stack	WeasyPrint, Jinja2, Python
Depends on	TASK 05
Description	Generate the complete SCA package as a single PDF: Cover Page + Form + Conditions + Appendix. Watermarks, cross-section page numbering, BGCC branding.
Files to create	<code>backend/services/pdf_service.py</code> <code>backend/templates/cover_page.html</code> <code>backend/templates/form_of_agreement.html</code> <code>backend/templates/conditions.html</code> <code>backend/templates/appendix.html</code> <code>backend/templates/base_pdf.css</code>

AI Prompt	You are building SAMS. TASK 06: Build the PDF generation service using WeasyPrint and Jinja2. Create PDF templates for all 4 documents. The Jinja2 context for rendering receives: agreement data, all field values (field_id → entered_value dict), appendix_config (which rows to show + extra notes). COVER PAGE: BGCC logo (top left), title 'SUBCONTRACT AGREEMENT', reference number, project name, subcontractor name, date, status watermark (DRAFT/FINAL/EXECUTED). FORM OF SUBCONTRACT: Render the master form HTML with [Insert] placeholders replaced by actual entered values. F01 replaces the date blank. F02 replaces [Insert Name]. Etc. for all F01-F08. APPENDIX: Render only the rows where show_in_appendix=true in appendix_config. Each row: Item Description (from appendix_row_label) Clause No. (from appendix_clause_ref) Information and Data (entered_value + admin_extra_note if present). CONDITIONS: Render the conditions HTML with C01-C13 values inserted at their [Insert] positions. WeasyPrint config: A4 size, 2cm margins all sides, BGCC header on every page, page numbers as 'Page X of Y'. Combine all 4 into one PDF using WeasyPrint's HTML concatenation. POST /api/pdf/{agreement_id}/generate — triggers generation, saves to /uploads/agreements/{reference_number}/draft_{timestamp}.pdf, records in pdf_outputs table. GET /api/pdf/{agreement_id}/preview — returns PDF as inline content.
-----------	--

TASK 07	Deviation Report Generator
Phase	Week 2, Day 10
Stack	Python, WeasyPrint, Jinja2
Depends on	TASK 05 → TASK 06
Description	Auto-generate the deviation report for any agreement: compares every entered field value vs master default. Columns: Clause Default Entered Change Type AI Risk (filled later by Task 10).
Files to create	backend/services/deviation_service.py backend/templates/deviation_report.html backend/routers/pdf.py (add endpoint)
AI Prompt	You are building SAMS. TASK 07: Build the Deviation Report generator. Query: for a given agreement_id, join agreement_field_values with master_fields to get (clause_number, field_label, default_value, entered_value, is_modified_from_default). Build a report table with columns: Clause Number Clause Title Master Default Value Entered Value Change Type Risk (placeholder — filled by AI Task 10). Change Type logic: if entered_value == default_value → 'Unchanged'. If default_value is null/empty and entered_value is not → 'Filled'. If entered_value != default_value and both have values → 'Modified'. Summary line at bottom: 'X clauses modified from default. Y blank fields filled. Z fields unchanged.' Render as HTML using Jinja2, convert to PDF with WeasyPrint. Store in deviation_reports table. API: GET /api/agreements/{id}/deviation-report — returns PDF inline or generates if not exists. POST /api/agreements/{id}/deviation-report/regenerate — force regenerate. Make the report always available — at any workflow stage.

TASK 08	Approval Workflow Engine
Phase	Week 3, Day 11–12
Stack	FastAPI, SQLAlchemy, aiosmtplib
Depends on	TASK 05 → TASK 06 → TASK 07
Description	4-step approval chain state machine. Mandatory comments on every return. Email notifications at every transition. Each role sees only their pending agreements.
Files to create	<code>backend/services/workflow_engine.py</code> <code>backend/routers/workflow.py</code> <code>frontend/src/pages/WorkflowReview.tsx</code> <code>frontend/src/components/WorkflowTimeline.tsx</code>
AI Prompt	You are building SAMS. TASK 08: Build the approval workflow engine. When Admin submits an agreement (POST /api/agreements/{id}/submit), create 4 workflow_steps: {step_order:1, role:project_director, status:pending}, {step_order:2, role:accounts, status:pending}, {step_order:3, role:operation_manager, status:pending}, {step_order:4, role:gm, status:pending}. Set agreement.current_status = under_internal_review. Only step 1 is active at first — steps 2/3/4 are created but inactive. API endpoints: GET /api/workflow/pending — returns agreements where current reviewer's step is pending (role-filtered). POST /api/workflow/{step_id}/approve — marks step as approved, activates next step. If last step (GM) approved: set agreement.gm_approval_date = today, current_status stays under_internal_review until Admin generates PDF. POST /api/workflow/{step_id}/return — REQUIRES body {comment_text, clause_reference?}. Validation: comment_text cannot be empty. Creates workflow_comment. Sends email to Admin with: agreement reference, step name, who returned it, the comment, clause reference. Admin sees the comment when editing the agreement. POST /api/agreements/{id}/resubmit — Admin resubmits after fixing. Reactivates the SAME step that returned it (not step 1). Build React WorkflowReview page: show agreement summary + Deviation Report link + AI summary panel (placeholder) + list of all comments from all steps + approve/return buttons. WorkflowTimeline component: shows PD → Accounts → OM → GM with checkmarks/pending/returned indicators and timestamps.

TASK 09	Collaborative Comment System
Phase	Week 3, Day 13
Stack	FastAPI, React
Depends on	TASK 08
Description	All reviewers can see and edit all comments. Every edit tracked in comment_edit_history. Comment states: open/in_discussion/resolved. Admin resolves after acting.

Files to create	<code>backend/routers/comments.py</code> <code>frontend/src/components/CommentThread.tsx</code>
AI Prompt	You are building SAMS. TASK 09: Build the collaborative comment system. Requirements: (1) Any reviewer can edit any existing comment, regardless of who wrote it. (2) Every edit is saved to <code>comment_edit_history</code>: {<code>comment_id</code>, <code>edited_by</code>, <code>previous_text</code>, <code>new_text</code>, <code>edited_at</code>}. (3) The <code>workflow_comments</code> table stores: <code>original_author_id</code> (never changes), <code>last_edited_by_id</code> (updates on each edit), <code>comment_text</code> (always the latest version), <code>last_edited_at</code>. (4) API: GET <code>/api/agreements/{id}/comments</code> — returns all comments for this agreement across all steps, ordered by <code>created_at</code>. Each comment includes: <code>original_author</code> name, <code>created_at</code>, <code>comment_text</code> (latest), <code>last_edited_by</code> name, <code>last_edited_at</code>, <code>status</code>, <code>edit_history</code> array. PUT <code>/api/comments/{id}</code> — edit comment text. Body: {<code>comment_text</code>}. Saves previous text to <code>comment_edit_history</code>. PATCH <code>/api/comments/{id}/status</code> — change status (<code>open</code>/<code>in_discussion</code>/<code>resolved</code>). <code>Resolved</code> can only be set by Admin. (5) React <code>CommentThread</code> component: shows each comment as a card with author badge, timestamp, text. If edited: shows 'Edited by [name] [time]' and an expandable 'View edit history' link. Edit button visible to all authenticated users. Status badge per comment. (6) Email notification when a comment is modified: notify the original comment author that their comment was edited, by whom, and what it was changed to.

TASK 10	AI Integration Layer
Phase	Week 5, Day 18–19
Stack	Python, OpenAI GPT-4o, Redis
Depends on	TASK 07 → TASK 08
Description	5 AI functions, all results cached in Redis. Every AI output requires human confirmation — no auto-actions. Role-specific summaries.
Files to create	<code>backend/services/ai_service.py</code> <code>backend/routers/ai.py</code> <code>frontend/src/components/AIReviewPanel.tsx</code>

AI Prompt

You are building SAMS. TASK 10: Build the AI integration layer using OpenAI GPT-4o. Create backend/services/ai_service.py with 5 async functions. All functions cache results in Redis with key pattern sams:ai:{agreement_id}:{review_type}. TTL = 24 hours. Function 1 — compare_clauses(agreement_id): get deviation report data (clause, default, entered, change_type). Send to GPT-4o with prompt: 'You are a legal contract analyst for a UAE construction company. Review these clause modifications to a subcontract agreement. For each modified clause, identify: what changed, potential legal implication. Return JSON: [{clause_number, clause_title, change_summary, implication}]'. Function 2 — detect_risks(agreement_id): use clause comparison output. Prompt: 'Rate the legal and financial risk of each modification: 1=none, 2=low, 3=medium, 4=high, 5=critical. Explain in plain English. Return JSON: [{clause_number, risk_score, risk_explanation}]'. Function 3 — generate_summary(agreement_id, role): role-specific. For accounts: focus on payment terms, advance, retention, securities, LD rates. For operation_manager: scope, programme, milestones, DLP. For gm: 5-line executive brief + top 3 risk items. Function 4 — suggest_responses(sheet_id): get all comments_resolution_sheets rows. For each: prompt 'Draft a professional contractual response to this subcontractor comment based on the agreement terms. Be firm but professional.' Return suggested_response per row. Function 5 — validate_revision(agreement_id, sheet_id): after Admin revises, check if each comment in the resolution sheet was addressed. Return: [{comment_id, is_addressed(bool), explanation}]. API: POST /api/ai/{agreement_id}/analyze (triggers 1+2), GET /api/ai/{agreement_id}/summary?role=gm, POST /api/ai/resolution/{sheet_id}/suggest, POST /api/ai/{agreement_id}/validate-revision. All endpoints return {status:success, data:{...}, cached:bool}. Note: all outputs are suggestions — frontend must show a confirmation step before saving.

TASK 11	Comments Resolution Cycle
Phase	Week 4, Day 14–15
Stack	FastAPI, React
Depends on	TASK 09 → TASK 10
Description	Subcontractor comments cycle. Admin creates sheet. PD reviews. OM confirms. Shortened re-approval (OM then GM only). Execution locking when signed.
Files to create	backend/routers/resolution.py backend/services/resolution_service.py frontend/src/pages/CommentsResolution.tsx

AI Prompt

You are building SAMS. TASK 11: Build the subcontractor comments resolution cycle. Admin records subcontractor response: PATCH /api/agreements/{id}/subcontractor-response with body {response_type: 'signed'|'comments', signed_scan_path?: string, comments_count?: number}. If signed: set execution_date=today, is_executed=true, current_status=completed, status_updated_on=now. Lock the agreement (no further edits to field values). If comments: set current_status=under_bgcc_revision. POST /api/agreements/{id}/resolution-sheet — Admin creates the sheet. Body: {items: [{subcontractor_comment, clause_reference}]}. System calls AI Task 10 function 4 to pre-fill ai_suggested_response. GET /api/resolution/{agreement_id} — returns all resolution sheet rows with all responses (ai, pd, om, final). PUT /api/resolution/{agreement_id}/items/{item_id} — update any response field. All reviewers (PD, OM, Admin) can update any response. Track last_edited_by. Shortened approval: POST /api/resolution/{agreement_id}/submit-for-approval — creates 2 workflow_steps: {step_order:1, role:operation_manager}, {step_order:2, role:gm}. After both approve, Admin can send revised agreement. Build React CommentsResolution page: table of subcontractor comments. Each row shows: comment | clause ref | AI suggestion | PD response | OM response | final response | resolved checkbox. All response fields editable by authorized users. Progress indicator showing OM/GM approval status.

TASK 12	Records Archive
Phase	Week 4, Day 16–17
Stack	FastAPI, React
Depends on	TASK 05 → TASK 11
Description	Project-wise + subcontractor-wise archive. Current Status field (8 values) + Status Updated On. Executed agreements locked and immutable. Excel export.
Files to create	backend/routers/archive.py frontend/src/pages/Archive.tsx

AI Prompt	You are building SAMS. TASK 12: Build the records archive. API: GET /api/archive/projects/{project_id} — returns all agreements for a project. Each agreement record includes: id, reference_number, subcontractor_name, current_status, status_updated_on, gm_approval_date, execution_date, is_executed, created_at. GET /api/archive/subcontractors/{subcontractor_id} — returns all agreements with a subcontractor across all projects. Same fields plus project_name and project_code. Both endpoints support filters: status, date_from, date_to. Both support search: by reference_number. GET /api/archive/agreements/{id} — get full agreement detail (view-only if is_executed=true). GET /api/archive/agreements/{id}/download — download the final executed PDF. GET /api/archive/export?project_id={id} OR ? subcontractor_id={id} — export list as Excel using openpyxl. Columns: Reference No, Project, Subcontractor, Status, GM Approval Date, Execution Date. Status values (8): Under Drafting Under Internal Review Draft Forwarded to Subcontractor Under Subcontractor Review Under Subcontractor Signature Under BGCC Revision Under GM Signature Completed. status_updated_on: auto-update trigger on every status change in the agreements table (use SQLAlchemy event listener or update in every status-changing endpoint). React Archive page: tabs for Project View and Subcontractor View. Table with all columns + status badge (color-coded: amber=in-progress, green=completed, red=revision). Filter panel on the left. Export to Excel button.
-----------	---

TASK 13	User Management + Admin Dashboard
Phase	Week 5, Day 20
Stack	React, FastAPI
Depends on	TASK 03 → TASK 12
Description	BGCC manages their own users — Admin UI to add/edit/deactivate/change roles. Agreement status dashboard with filters. Audit trail viewer.
Files to create	frontend/src/pages/UserManagement.tsx frontend/src/pages/Dashboard.tsx backend/routers/reports.py
AI Prompt	You are building SAMS. TASK 13: Build user management and the admin dashboard. USER MANAGEMENT (Admin only): React page at /users. Table of all users: name, email, role badge, status (active/inactive), last login, actions (edit, deactivate). Add User modal: name, email, role dropdown, auto-generate temp password. Edit User: change name, email, role. Deactivate: soft delete (is_active=false). All changes logged to audit_log. API already built in Task 03 — wire up the frontend. ADMIN DASHBOARD at /dashboard: (1) Summary cards: Total Agreements, Under Review, With Subcontractor, Completed This Month. (2) Agreements table: all agreements with reference, project, subcontractor, status badge, status_updated_on, actions (view, generate PDF, download). Filter by: status, project, subcontractor, date range. Search by reference number. (3) Audit trail viewer: table of audit_log — who, what action, on what entity, when. Filter by user, action type, date. Paginated. (4) Master template versions panel: shows current active version per type (form/conditions/ appendix) with 'Manage Versions' button linking to MasterTemplates page.

TASK 14	Security + Monitoring + Launch
Phase	Week 6, Day 21–24
Stack	Nginx, Certbot, slowapi, bash, Supervisor
Depends on	TASK 13
Description	HTTPS, rate limiting, input sanitization, daily backups, error handling, performance verification, go-live.
Files to create	<code>nginx/sams.conf</code> (update) <code>scripts/backup.sh</code> <code>backend/middleware/security.py</code> <code>supervisord.conf</code> (update)
AI Prompt	You are building SAMS. TASK 14: Secure and launch the system. (1) HTTPS: run 'certbot --nginx -d yourdomain.com' and add auto-renewal cron. (2) Nginx security headers: add X-Frame-Options: DENY, X-Content-Type-Options: nosniff, X-XSS-Protection: 1; mode=block, Strict-Transport-Security: max-age=31536000. (3) Rate limiting: apply slowapi limiter to all /api/ routes. Default: 100 requests/minute per IP. Stricter on auth routes: 10/minute. (4) Input sanitization: add a Pydantic validator that strips HTML tags from all text fields. Use bleach library for sanitization. (5) Error handling: create a global exception handler in FastAPI that returns {error: error_code, message: user_friendly_message, timestamp} for all unhandled exceptions. Log full stack trace to file. (6) Backup script: /scripts/backup.sh — pg_dump sams_db → /backups/sams_{date}.sql.gz. Keep last 30 days. Add as daily cron at 2am. (7) Supervisor config: FastAPI process with autorestart=true, stdout_logfile=/var/log/sams/app.log, stderr_logfile=/var/log/sams/error.log. (8) Performance: verify the system handles 15 concurrent users with < 2 second response time on all main pages. Use Apache Bench (ab) for basic load testing. (9) Create all 15 user accounts per the client-provided list. (10) Seed all 3 master templates from the client's actual documents (File 01, File 02, File 03).

